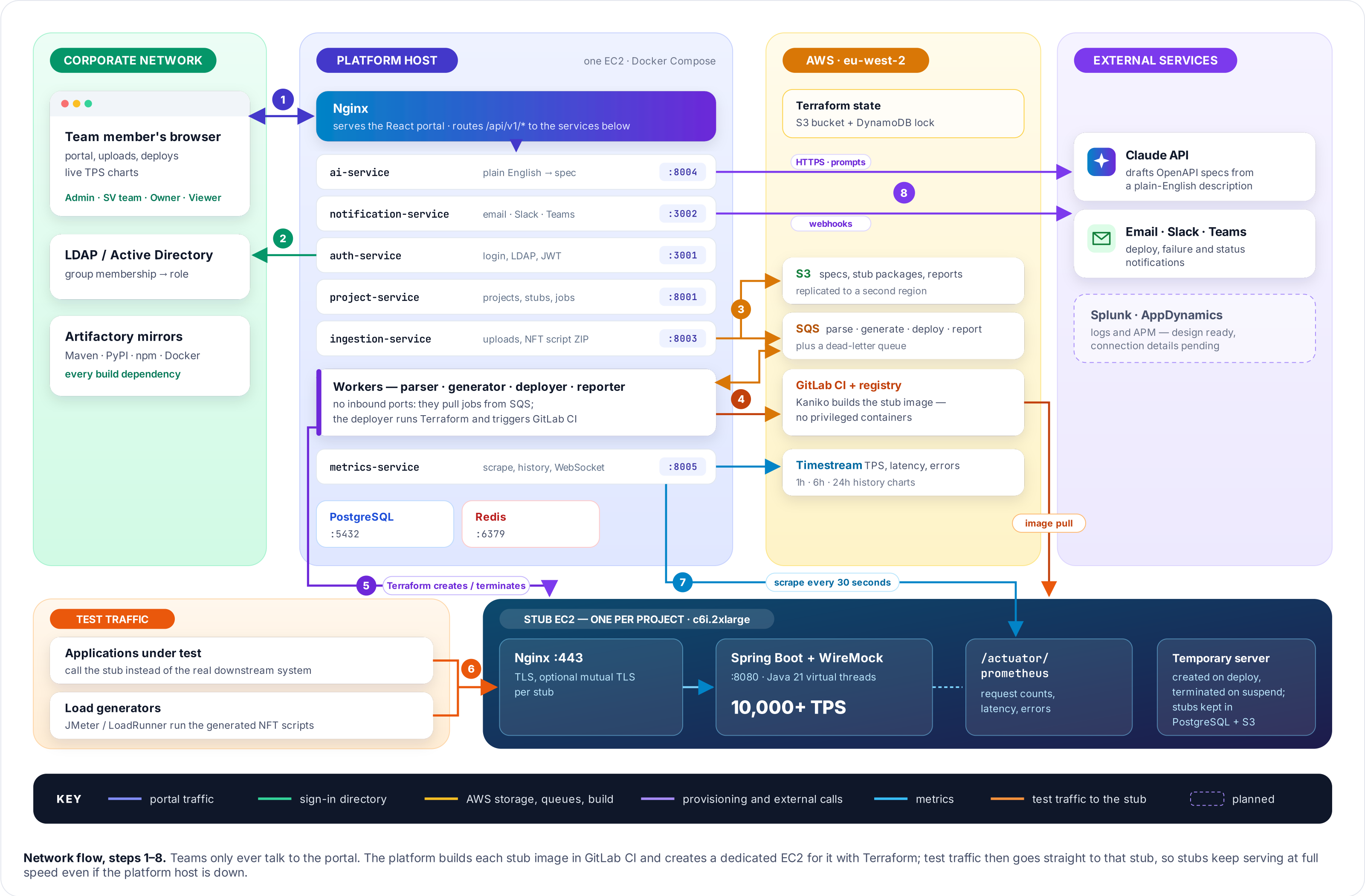


Mockingbird — network flow

Which machine talks to which, and in what order — from a team signing in, through building and provisioning a stub, to applications and load generators sending test traffic to it.



The flow, step by step

- 1 Use the portal.** The browser loads the React portal from Nginx, which routes every `/api/v1` call to the right service. Live TPS arrives over a WebSocket.
- 2 Sign in.** The auth service checks the user against LDAP / Active Directory and maps their group to a role (admin, SV team, owner, viewer).
- 3 Upload a spec.** The ingestion service stores the file in S3 and puts a parse job on an SQS queue.
- 4 Parse, generate, build.** Workers take jobs from SQS in turn. The deployer starts a GitLab CI pipeline where Kaniko builds the stub image from Artifactory dependencies.
- 5 Provision.** The deployer runs Terraform to create the project's EC2, which pulls the image and starts the stub. The team gets a URL and API key.
- 6 Test.** Applications under test and load generators call the stub on HTTPS 443 (mutual TLS optional), never the real dependency.
- 7 Monitor.** The metrics service scrapes each stub every 30 seconds into Timestream and pushes live figures to the portal.
- 8 AI and alerts.** The AI service asks Claude to draft specs; the notification service posts to email, Slack and Teams.

Ports and protocols

FROM → TO	PORT / CHANNEL	CARRIES
Browser → Nginx	HTTP(S)	portal, REST <code>/api/v1</code> , WebSocket
Nginx → services	3001–8005	internal Docker network only
Auth → directory	LDAP	sign-in and group lookup
Services → data	5432 6379	PostgreSQL, Redis
Platform → AWS	HTTPS	S3, SQS, Timestream, EC2 APIs
Tests → stub	443 / 8080	TLS / mTLS, or plain HTTP
Metrics → stub	/actuator/prometheus	every 30 seconds

Resilient by design: stubs run on their own EC2s, separate from the platform. If the platform host is unavailable, the portal pauses but every live stub keeps serving test traffic.